# Cross validation

Resampling methods involve drawing samples from a training set and refitting the model of interest on each sample to obtain information about the model that is fitted on the complete training set. Cross-validation is one of the most commonly used resampling methods.

## The validation set

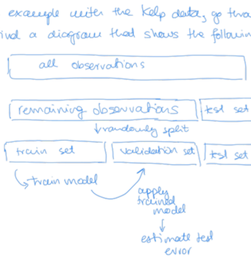
Goal: estimating the test error associated with fitting a model on a training set.

Q: this is the principle I don't necessarily get: we want to estimate the error rate on the test set, we don't need to estimate it, just train and compute it. Why go through cross-validation for this?
I understand cross-validation much better when used to fine tune a parameter.
OK. Let's keep going.

Reasons why we'd want to do this (without computing the test error directly):
fill out.

Approach:
1) randomly divide the available set of observations into training set and validation or hold-out set.
2) The model is fit on the training set
3) the fitted model is used to predict the responses on the validation set.
4) The validation set error provides an estimate of the test error.

Make an example with the Kaggle data, go through steps 1-4.
Create / find a diagram that shows the following:




Q: I need to get this workflow correct with how it is implemented in practice.
When we divide our data into training and validation set, the test error estimate depends on the particular randomly selected validation set. Different validation sets can greatly vary this estimate.

Show examples of validation error computed with different validation sets.
Q: I don't understand if your final model is the one you fit with the training set only or you K-integrate the validation set into the training set. Probably for the validation set approach it is only trained on the train set.

## Leave-one-out cross validation

Leave-one-out cross validation (LOOCV)

I will only explain this with diagrams, there must be some good ones online I can use.
The observations in the training set are $\{(x_2, y_2), \dots, (x_n, y_n)\}$
Approach:
1) "Hold" the 1st observation in the training set $(x_1, y_1)$
2) Fit the model on the remaining observations $\{(x_2, y_2), \dots, (x_n, y_n)\}$
3) Apply the fitted model to make a prediction for $y_1$ using $x_1$:
predicted value: $\hat{y}_1$
4) Calculate the error rate:
$MSE_1 = (y_1 - \hat{y}_1)^2$ for quantitative
$E_1 =$ ? for categorical
5) Repeat steps 1-4 now holding out the second observation $(x_2, y_2)$ and computing the error rate
$MSE_2 = (y_2 - \hat{y}_2)^2$
6) Repeating this approach $n$ times produces $n$ errors
$MSE_1, \dots, MSE_n$.

The LOOCV estimate for the test error is the average of these test error estimates:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^{n} MSE_i .$$

* This needs to be adjusted to explain with categorical variables to follow the Kaggle example.

Advantages:
- The $n$ models are fit using most of the training data: we only leave one observation out when we are training the $n$ submodels
- There is no randomness in doing a train / validation sets split, so the LOOCV always produces the same estimate.
- Can be used with any kind of predictive model.

Potential drawbacks:
- Can be expensive to implement since the model has to be fit $n$ times. Time consuming if $n$ is large.

If the model we are evaluating is a linear model fitted by least squares, then there are formulas to directly calculate it without fitting.

## K-fold cross-validation

Alternative to LOOCV.

Approach:
1) Randomly divide the training set into $K$ groups or folds of approximately equal size
2) First fold is validation set, model is fit on remaining $k-1$ folds.
3) Error is computed on the hold out fold: $MSE_1$
4) Repeat steps 2+3 with the rest of the folds and obtain $K$ estimates of the test error $MSE_1, \dots, MSE_K$.
5) Obtain the $K$-fold CV estimate by averaging those values

$$CV_{(k)} = \frac{1}{K} \sum_{i=1}^{k} MSE_i$$

In practice we usually perform $K$-fold cross validation using $K=10$ or $K=5$.
$K$-fold has computational advantages over LOOCV.
Show an equivalent of figure 5.6. where cross-validation error rates across models with different flexibility are compared and then the model with the minimum CV error is marked.
We are interested in the location of the minimum point in the estimated MSE curve. We use CV on a number of different ML models or on a single model with different levels of flexibility, in order to identify the model with the minimum test errors.

## Cross-validation in classification problems.

Instead of using MSE to quantify test error, we use the number of misclassified observations.

In LOOCV the error rate is $CV_{(n)} = \frac{1}{n} \sum_{i=1}^{n} Err_i$, where $Err_i = \begin{cases} 1 & \text{if } \hat{y}_i = y_i \\ 0 & \text{if } \hat{y}_i \neq y_i \end{cases}$.

The $K$-fold CV error rate are defined similarly.
(?) Actually not sure how this would look like when computed on a fold with $K$ elements. Is $Err_i$ = fraction of observations correctly classified in the fold
or
$Err_i$ = total number of observations correctly classified in the fold.

We need to have an exercise in the homework or lab about fitting polynomial models using least squares. And then extending this into having polynomial functions of the predictors in logistic regression to obtain near linear boundaries.

* We will do the CV examples with KNN

## CV to select classification threshold.